

Guión de defensa del TFM

Desarrollo de una aplicación de gestión de planes de boda Laura Rosón Lavín · Tutor: Juan José Ramos
Muñoz Máster en Ingeniería Informática · Universidad de Granada

21 diapositivas · 20 minutos

Cómo usar este documento

1. **La idea central** — lo único que el tribunal recordará al salir.
2. **El proyecto en 10 líneas** — para resumirlo si te lo piden de golpe.
3. **Guión diapositiva a diapositiva** — qué decir y cuánto dura cada una.
4. **Fundamentos técnicos** — *la parte más importante*. Entiende el porqué, no memorices.
5. **Preguntas** — resumen. El banco completo está en `preguntas-defensa.pdf`.

Consejo: la exposición se ensaya; las preguntas no se memorizan. Dedica el 70 % del estudio a la parte 4.

Dos avisos sobre el tono

El tribunal no es especialista. Tu tutor lo ha dicho expresamente: hay cosas que se le escapan. Por eso las diapositivas están redactadas en lenguaje llano y el guión también. Cuando uses un término técnico, **explícalo en la misma frase**: «Prisma, que es el traductor entre mi código y la base de datos...». Nunca sueltes la palabra sola.

Cada tecnología va con su porqué. No enumeres herramientas: di qué es, para qué sirve y por qué la elegiste. Es lo que demuestra que hay criterio detrás.

1. La idea central

He construido una aplicación que centraliza toda la organización de una boda, y le he añadido asistencias de inteligencia artificial que proponen tareas, distribución de mesas y reparto de presupuesto. La clave de diseño es que la IA nunca decide: propone, el sistema verifica que la propuesta sea válida, y la pareja confirma.

1. **El hueco:** las plataformas actuales usan la IA para *su* negocio —recomendar proveedores—, no para las decisiones internas de la pareja.
2. **La aportación:** un patrón de *IA verificada y bajo control del usuario*.
3. **El resultado:** aplicación completa, ocho objetivos cumplidos, 18 pruebas e integración continua.

2. El proyecto en 10 líneas

Organizar una boda implica coordinar durante meses invitados, mesas, tareas, presupuesto y proveedores, y todo está relacionado entre sí. Las herramientas que existen son buenas, pero viven de un modelo de *marketplace*: su objetivo es venderte proveedores. He desarrollado una aplicación autocontenida donde una pareja gestiona todo desde un único panel: invitados y acompañantes con confirmación de asistencia, grupos y mesas, tareas y agenda, presupuesto y proveedores, y un panel de seguimiento con avisos. Encima he integrado tres asistencias de IA que proponen el listado de tareas, la distribución de mesas y el reparto del presupuesto. Está hecha con React en el cliente, Node.js con Express en el servidor y PostgreSQL con Prisma. Lo verifiqué con 18 pruebas automatizadas, integración continua y un conjunto de 64 invitados.

3. Guión diapositiva a diapositiva

Duración total: 20:00 exactos.

Bloque	Diapositivas	Tiempo
Apertura	1-2	0:50
1 · El problema	3	1:10
2 · Estado del arte y objetivos	4-5	1:55
3 · Diseño	6-13	9:45
4 · Implementación	14-16	2:55
5 · Evaluación y resultados	17-18	1:55
6 · Conclusiones y líneas futuras	19-20	1:20
Cierre	21	0:10

Antes de empezar: mira al tribunal y pide permiso. *«Buenos días. Con el permiso del tribunal, comienzo mi exposición.»*

1 — Portada · 0:25

«Buenos días. Soy Laura Rosón Lavín y voy a presentar mi Trabajo Fin de Máster, *Desarrollo de una aplicación de gestión de planes de boda*, dirigido por el profesor Juan José Ramos Muñoz.»

Transición: «La exposición se estructura en seis partes.»

2 — Tabla de contenidos · 0:25

«Empezaré por el problema que motiva el trabajo; veremos qué soluciones existen y qué hueco dejan; después el diseño de la solución, **que es la parte central y a la que dedicaré la mitad de la exposición, capa por capa**; luego la implementación, la evaluación, y terminaré con las conclusiones y las líneas futuras.»

Transición: «Empecemos por el problema.»

3 — El problema · 1:10

(Señala primero las tres cifras, luego pasa la mano por las etiquetas.)

«Organizar una boda no es una tarea: es un proyecto de entre ocho y doce meses.

Según el estudio anual de The Knot Worldwide, el *Real Weddings Study* de 2026, una boda implica de media unos 117 invitados y la contratación de unos 13 profesionales distintos.

Pero el problema no es solo el volumen, sino **la cantidad de cosas distintas que hay que llevar a la vez**: la lista de invitados y sus acompañantes, quién ha confirmado y quién no, las alergias y los menús especiales de cada uno, los grupos, el reparto en mesas y asientos, las tareas con sus plazos, los eventos de la agenda, las partidas de gasto y los contratos de cada proveedor.

Y todo eso **depende entre sí**: el número de invitados condiciona las mesas, las mesas condicionan el catering y el catering condiciona el presupuesto. Si cambias una cosa, cambian las demás.

En la práctica, esa información acaba dispersa en hojas de cálculo, cuadernos y aplicaciones separadas, y mantener una visión de conjunto coherente se vuelve costoso y propenso a errores.»

Transición: «Soluciones digitales ya existen. Veamos qué ofrecen y qué les falta.»

4 — Estado del arte · 1:05

(Señala la última fila de la tabla al llegar a la IA.)

«Analicé las tres plataformas más representativas: Bodas.net, referente en España, y The Knot y Zola, líderes internacionales.

Las tres cubren bien la organización básica. Pero las tres se construyen sobre el mismo modelo de negocio: son *marketplaces*, sus ingresos vienen del directorio de proveedores y las listas de regalos, y las herramientas de organización son un complemento de ese negocio. De hecho, algunas funciones concretas, como el plano de mesas de Zola, son de pago.

Y aquí está la observación clave: **la inteligencia artificial que han empezado a incorporar se orienta a su negocio**. The Knot la usa para recomendar proveedores y Zola para redactar textos de agradecimiento. Ninguna la aplica a las **decisiones organizativas** de la pareja.

¿Y a qué me refiero con decisiones organizativas? A las tres que resuelve mi trabajo: **decidir qué tareas hay que hacer y en qué mes**, retrocediendo desde la fecha de la boda; **decidir quién se sienta con quién y en qué mesa**, respetando los grupos y la capacidad; y **decidir cómo se reparte el presupuesto** entre las distintas categorías de gasto. Son decisiones repetitivas, que consumen mucho tiempo y en las que un borrador de partida ahorra muchísimo trabajo. Ese es el hueco que detecté.»

Transición: «De ese hueco salen los objetivos del trabajo.»

5 — Objetivos · 0:50

«El objetivo general era desarrollar una aplicación que organizase de forma integral y centralizada todos los aspectos de una boda, incorporando asistencias de IA para las decisiones más repetitivas.

Lo agrupé en tres objetivos. El primero, **gestión integral**: cubrir en una sola herramienta todo el ciclo, desde la cuenta y las bodas hasta los invitados, las mesas, las tareas, el presupuesto y el seguimiento. El segundo, **las asistencias con inteligencia artificial**, que es el elemento diferenciador. Y el tercero, **la calidad del producto**: seguridad, usabilidad, robustez y pruebas.

Cada uno se desglosa en objetivos específicos —ocho en total— que en la memoria se traducen en requisitos concretos y cuyo cumplimiento se evalúa uno a uno.»

 **Si te preguntan por qué ocho:** «Los ocho objetivos específicos son el desglose operativo de estos tres. Los presento agrupados porque a este nivel lo relevante son las tres metas; el detalle está en la memoria, donde cada uno se enlaza con sus requisitos y sus pruebas.»

Transición: «Tras ver qué hace falta, vamos a ver cómo lo he diseñado. Esta es la parte central de la exposición.»

6 — Arquitectura · 1:00

(Recorre las tres bandas con la mano, de arriba abajo.)

«La aplicación se organiza en tres capas, y cada una tiene una responsabilidad clara.

La **capa de presentación es el cliente**: lo que se ejecuta en el navegador, lo que el usuario ve y toca. Está hecha con React.

La **capa de aplicación es el servidor**: donde está la lógica, la que decide qué se puede hacer y qué no. Está hecha con Node.js y Express.

Y la **capa de persistencia es la base de datos**: donde todo queda guardado. Es PostgreSQL.

El cliente le pide datos al servidor, y el servidor consulta y guarda en la base de datos. Nunca se salta un paso: el navegador no habla directamente con la base de datos.

Aparte quedan los **servicios externos** —la inteligencia artificial y el envío de correo—, que se conectan **solo con el servidor**. El cliente nunca ve sus credenciales.

A continuación voy a dedicar una transparencia a cada capa, explicando qué tecnología usa, cómo está diseñada y por qué la planteé así.»

Transición: «Empezando por la capa que ve el usuario.»

7 — Capa de presentación · 1:20

(Ve fila por fila. No leas la tabla entera: destaca las tres marcadas.)

«En el cliente uso cinco tecnologías, y quiero explicar brevemente qué hace cada una.

React es la biblioteca con la que construyo la interfaz a base de piezas reutilizables: una tarjeta de invitado, un diálogo... Lo elegí por tener el modelo más sencillo y el ecosistema más amplio; gracias a eso encontré bibliotecas ya maduras para el calendario y las gráficas.

TypeScript es JavaScript al que se le añaden tipos de datos. Su ventaja es que me avisa de los errores mientras escribo, no cuando la aplicación ya está funcionando.

Las tres siguientes son menos conocidas, así que me detengo un poco más.

Vite es la herramienta que compila y sirve la aplicación mientras se desarrolla. La elegí porque arranca y refresca los cambios casi al instante; con las herramientas clásicas, cada cambio tardaba varios segundos en verse, y eso multiplicado por cientos de cambios al día es mucho tiempo perdido.

Tailwind CSS es un sistema de estilos mediante clases ya definidas: en lugar de escribir hojas de estilo a medida para cada pantalla, compongo el aspecto con piezas predefinidas. Eso me dio coherencia visual en toda la aplicación sin esfuerzo.

Y **shadcn/ui** es una colección de componentes ya accesibles —botones, diálogos, menús— que **se copian dentro del proyecto** en lugar de instalarse como una dependencia externa. La ventaja es que puedo adaptarlos libremente y no arrastro una biblioteca pesada con una estética impuesta.

Y la decisión de diseño más importante de esta capa: **todas las llamadas de red pasan por un único punto**, que añade la sesión, la renueva cuando caduca y unifica los errores. Así no repito esa lógica en cada pantalla y el comportamiento es idéntico en toda la aplicación.»

Transición: «La seguridad real, sin embargo, está en la siguiente capa.»

8 — Capa de aplicación · 1:20

(Recorre la cadena de tres pasos con la mano.)

«Cada petición que llega del cliente recorre siempre el mismo camino, y son tres pasos.

Primero, las **rutas de la API REST**, que son las «puertas» del servidor: una dirección para los invitados, otra para las tareas, otra para el presupuesto.

Segundo, el **controlador**, que comprueba que los datos que llegan son correctos **antes de tocar nada**: que el nombre no venga vacío, que la fecha sea una fecha.

Y tercero, el **servicio**, que es donde están las reglas del negocio y donde se habla con la base de datos.

En cuanto a tecnologías: **Node.js** me permite ejecutar JavaScript en el servidor, lo que significa que uso el mismo lenguaje en cliente y servidor; trabajando sola, eso reduce mucho el cambio de contexto. **Express** es la estructura mínima para crear esas rutas: la elegí por madura y simple, frente a alternativas como NestJS que imponen mucha estructura que aquí no compensa. Y **Zod** define qué forma deben tener los datos que entran; su ventaja es que con una sola definición valida los datos **y** genera los tipos, así que no pueden desincronizarse.

¿Por qué separar «comprobar» de «hacer»? Porque me permite **probar la lógica sin levantar un servidor**, y porque si mañana cambio la forma de validar, no toco las reglas del negocio.»

Transición: «Bajemos al último nivel, donde viven los datos.»

9 — Capa de persistencia · 1:05

(Señala los tres bloques de izquierda a derecha.)

«Aquí hay dos piezas que conviene no confundir.

PostgreSQL es la base de datos: el programa que guarda de verdad la información en disco y garantiza que sea coherente.

Prisma es el intermediario entre mi código y PostgreSQL. Cuando en mi programa escribo «dame los invitados de esta boda», Prisma traduce esa orden al lenguaje que entiende la base de datos y, de paso, comprueba que los campos que pido existen realmente.

¿Qué me aporta? Que el modelo de datos se declara **una sola vez** en un fichero, y a partir de ahí Prisma genera automáticamente el código de acceso y las **migraciones**, que son el registro de cada cambio de la estructura de la base de datos. Eso hace que los cambios sean repetibles: puedo montar la base de datos desde cero en otro ordenador y quedará exactamente igual.

Y la consecuencia práctica más importante: si me equivoco en el nombre de un campo, **falla al compilar**, no delante del usuario.

Por último, una decisión deliberada: las garantías de integridad —que un asiento no se asigne dos veces, que al borrar una boda no queden restos— las impone **PostgreSQL**, no mi código. Una restricción de la base de datos no se puede eludir; una comprobación escrita en el código, sí, en cuanto añades una vía nueva que se olvida de hacerla.»

Transición: «Veamos cómo está organizada esa información.»

10 — Modelo de datos · 0:55

(Señala «Boda», en el centro del diagrama, y desde ahí ve abriendo hacia las entidades.)

«Este es el modelo de datos, y tiene una forma muy característica: **todo cuelga de la boda**. Un usuario tiene una o varias bodas, y de cada boda cuelgan los invitados, los grupos, las mesas, los eventos, las tareas, el presupuesto y los proveedores.

Esa forma no es casual, es la que permite el **aislamiento**: como cada dato sabe a qué boda pertenece, puedo acotar cualquier consulta al propietario y garantizar que nadie vea bodas ajenas.

Tres detalles. Los **acompañantes** son invitados que apuntan a su invitado principal, lo que evita duplicar una tabla casi idéntica. Para que **un asiento no se ocupe dos veces** hay una restricción sobre la pareja mesa-asiento. Y los borrados se propagan en cascada, así que al eliminar una boda no quedan restos sueltos.»

Transición: «Fuera de estas tres capas quedan los servicios externos.»

11 — Integraciones externas · 1:05

(Señala la línea resaltada del código y luego el texto de qué se envía.)

«El sistema se integra con dos servicios externos: la inteligencia artificial y el envío de correo.

A la izquierda está la llamada real a la IA. Es **la única función de todo el proyecto que habla con ella**, y vive solo en el servidor. Le indico el modelo que quiero usar, una temperatura baja —0,4— porque busco respuestas consistentes y no creativas, y le paso dos mensajes: unas reglas de comportamiento y **el contexto de la boda**, que es lo que construyo a partir de la base de datos.

Sobre los datos: se envían la fecha, el número de invitados y, en el caso de las mesas, los nombres y los grupos. **No se envían correos, ni teléfonos, ni contraseñas.**

Para el correo, si hay un servidor configurado envía de verdad, y si no, usa una cuenta de pruebas que no entrega nada pero devuelve un enlace de vista previa. Eso me permitió desarrollar y probar todo el flujo de invitaciones sin montar infraestructura de correo.

¿Por qué aisladas? Porque cada servicio externo vive en un único fichero: cambiar de proveedor no me obliga a tocar el resto, y un fallo suyo no tumba la aplicación.»

 **Prepárate esta pregunta, es muy probable:** «¿Envía datos personales a OpenAI?» «Sí, en la distribución de mesas se envían nombres y grupos, que son datos personales. Es una consideración de privacidad real. Se manda el mínimo necesario —sin correos ni teléfonos— y siempre desde el servidor. En un despliegue real habría que informarlo expresamente en la política de privacidad y, mejor aún, seudonimizar: enviar identificadores en lugar de nombres. Lo asumo como una limitación y una mejora pendiente.»

Transición: «Y hay una preocupación que atraviesa todas las capas: la seguridad.»

12 — Seguridad · 1:10

(Señala primero las dos cajas de arriba, luego la fila resaltada de la tabla.)

«La seguridad no es una capa, afecta a todas, así que le dedico una transparencia.

Cuando inicias sesión, el servidor no te entrega una credencial sino **dos**, y cada una tiene un propósito distinto.

El **token de acceso** dura pocos minutos y viaja en cada petición. Al durar tan poco, si alguien lo interceptara, dejaría de servir casi inmediatamente.

El **token de refresco** dura días, pero solo sirve para pedir un token de acceso nuevo, y —esto es lo importante— **se guarda en la base de datos**. Como está guardado, se puede anular: eso es lo que hace cerrar sesión de verdad. Si solo tuviera un token, o lo hacía corto y obligaba al usuario a iniciar sesión cada diez minutos, o lo hacía largo y no podía revocarlo. Con dos consigo las dos cosas.

Sobre la autorización, hay **doble comprobación de dueño**: aunque alguien cambiara el identificador de la boda en la dirección, no vería nada.

Y quiero destacar una decisión concreta: cuando alguien intenta acceder a una boda que no es suya, el sistema responde **«no encontrado»**, no «prohibido». Puede parecer un detalle, pero si respondiera «prohibido» estaría confirmando que esa boda existe, y eso ya es información que le estoy dando a un atacante. Diciendo «no encontrado» no revelo nada.

Las contraseñas se guardan transformadas y no se pueden recuperar ni desde la propia base de datos. Y si las claves de seguridad del servidor no son seguras, **el servidor directamente no arranca**: prefiero que falle a que arranque mal.»

Transición: «Y con todo esto montado, llegamos a la aportación central del trabajo.»

13 — El diseño de la IA · 1:50 ★ *La más importante*

(Recorre los cinco pasos con la mano, leyendo el ejemplo de cada uno. Ve despacio.)

«El problema de meter un modelo de lenguaje dentro de una aplicación es que devuelve **texto**, y el texto no es fiable como dato: puede venir mal formado o inventarse valores. Si vas a guardar eso en una base de datos, tienes un problema serio.

Lo resolví con un flujo de cinco pasos, común a las tres asistencias. Lo explico con el ejemplo del asistente de tareas.

Uno, el contexto. El servidor construye un resumen con datos reales de la boda: «boda el 12 de junio de 2027, 64 invitados, 9 mesas, y estas 18 tareas ya creadas». Ese último dato es importante: así no me propone tareas que ya tengo.

Dos, el modelo. Se le llama exigiéndole la forma exacta que debe tener la respuesta: cada tarea debe traer título, categoría, prioridad y fecha. No le pido texto libre.

Tres, la validación. Aunque venga con esa forma, la compruebo igualmente: ¿están todos los campos?, ¿son del tipo correcto? Si algo falla, se descarta y no se usa.

Cuatro, la propuesta. Lo que se devuelve es un borrador editable. Por ejemplo: «Reservar el catering, categoría Banquete, prioridad Alta, ocho meses antes de la boda». La IA **nunca escribe en la base de datos**.

Y cinco, la confirmación. La pareja revisa la lista, desmarca lo que no le interesa, y solo entonces se guarda, pasando por los mismos puntos de alta que usa el resto de la aplicación.

Los dos principios que resumen todo esto son **fiabilidad del formato** y **control del usuario**. La IA es un asistente que propone, no un sistema que decide.»

Transición: «Veamos cómo se lleva esto al código.»

14 — El contrato con la IA · 1:00

(Señala las dos líneas resaltadas.)

«Cuando hablo de **contrato** me refiero a un acuerdo cerrado sobre la forma exacta que debe tener la respuesta: qué campos tiene que traer y de qué tipo es cada uno. Primero se impone y después se verifica, y son dos líneas de código.

Arriba, al llamar al modelo, le indico que la respuesta debe ajustarse a ese esquema de forma **estricta**.

Abajo, ya con la respuesta en la mano, la compruebo contra ese mismo esquema. Si no encaja, lanzo un error controlado y la propuesta no llega nunca al usuario.

Alguien podría preguntarse por qué verifico si el proveedor ya me lo garantiza. Porque **no quiero delegar la integridad de mis datos en un servicio externo**: si mañana cambian la API o algo falla, mi sistema no se rompe.

Y si la IA falla o no está configurada, **la aplicación avisa con un mensaje claro y todo lo demás sigue funcionando con normalidad**. No se bloquea nada: simplemente esa función no está disponible en ese momento.»

Transición: «Hubo un caso que no se resolvía solo con esto.»

15 — El caso difícil: las mesas · 1:00

(Recorre los cuatro pasos del flujo con la mano.)

«La distribución de mesas fue el caso más complicado, y merece la pena explicarlo porque muestra los límites de lo anterior.

El problema es que los identificadores de las mesas y de los invitados **se generan en cada boda**: no existen de antemano, así que no puedo fijarlos en el esquema. Eso significa que el contrato me garantiza la *forma* de la respuesta, pero no que los identificadores sean reales. El **modelo de lenguaje** podía devolverme una mesa que no existe, un asiento por encima de la capacidad, o sentar dos veces a la misma persona.

La solución fue añadir un paso intermedio. El modelo propone una asignación; el servidor la **depura**, comparándola con lo que hay realmente en la base de datos y descartando mesas inexistentes, asientos fuera de capacidad y repeticiones; solo la propuesta ya limpia se le enseña a la pareja; y cuando la acepta, **se aplica entera o no se aplica nada**, en un solo bloque.

La idea de fondo es la que resume la transparencia: nunca se confía en la salida del modelo, **se contrasta con lo que hay realmente en la base de datos.**»

Transición: «Y así queda la aplicación funcionando.»

16 — La aplicación en funcionamiento · 0:55

(Señala cada captura.)

«A la izquierda, el panel de seguimiento, que resume el estado de la boda: invitados por confirmación, ocupación de mesas, progreso de tareas, estado del presupuesto y la cuenta atrás hasta la fecha.

A la derecha, una **generación real** del asistente de tareas: cada propuesta con su categoría, su prioridad y su fecha, y las casillas que permiten descartar las que no interesen antes de confirmar.»

 **Opción de vídeo** (*tu tutor la sugirió*) Si grabas un vídeo de 2 minutos mostrando la aplicación, sustituye el texto anterior por:

«Con el permiso del tribunal, me gustaría mostrar un breve vídeo de dos minutos con el funcionamiento de la aplicación.»

(*Espera el gesto de aprobación antes de darle al play. Es lo que da imagen profesional.*)

Qué debe mostrar el vídeo, en este orden (no saltes de una cosa a otra al azar):

1. Panel de seguimiento y cuenta atrás (15 s)
2. Alta de un invitado con acompañante y alergias (25 s)
3. Mapa de mesas: asignar a alguien a un asiento (25 s)
4. **Generar tareas con IA: pulsar, ver la propuesta, desmarcar una y confirmar** (40 s)
5. Presupuesto con sus gráficas (15 s)

Cuenta el vídeo sin voz en off y ve narrando tú en directo: así controlas el ritmo y puedes parar si el tribunal pregunta.

Ajuste de tiempo si metes el vídeo (hay que recortar 1:20 en total):

Diapositiva	Sin vídeo	Con vídeo
4 · Estado del arte	1:05	0:50
7 · Capa de presentación	1:20	1:05
8 · Capa de aplicación	1:20	1:05
12 · Seguridad	1:10	1:00
16 · La aplicación	0:55	2:15
17 · Evaluación	1:00	0:50
18 · Funciones	0:55	0:40

Transición: «Pasemos a cómo lo he verificado.»

17 — Evaluación · 1:00

(*Señala la fila resaltada al hablar del aislamiento.*)

«Sobre el entorno: el servidor se ejecuta en Node.js contra una base de datos PostgreSQL levantada en un contenedor, las pruebas están hechas con Vitest y supertest, y todo se validó además con un conjunto de datos realista de 64 invitados.

Hay cinco niveles de comprobación. Las **unitarias** verifican piezas aisladas: el cifrado de contraseñas, las credenciales de sesión, la plantilla del correo y el tratamiento de errores. Las de **integración** recorren el camino completo de registro, creación de boda y alta de invitado, con peticiones reales contra la base de datos.

La tercera fila es la que más me interesa destacar: compruebo que **acceder a una boda ajena devuelve «no encontrado»**, verificando así el aislamiento del que hablaba antes.

Además se analiza el estilo y se compila tanto el cliente como el servidor, y todo esto se ejecuta **automáticamente con cada cambio** mediante integración continua, que funciona como red de seguridad: si algo se rompe, me entero en el momento.

En total, 18 pruebas automatizadas repartidas en cinco ficheros, todas superadas.»

Transición: «Veamos entonces qué hace la aplicación resultante.»

18 — Lo que hace la aplicación · 0:55

(Señala cada bloque; detente en el rojo.)

«Agrupando por áreas, la aplicación cubre lo siguiente.

En **invitados**: alta con acompañantes, confirmación de asistencia, alergias y menús, invitaciones por correo, una página pública donde el invitado confirma sin necesidad de cuenta, e importación y exportación en CSV.

En **organización**: grupos, el mapa de mesas con asignación de asientos, las tareas con prioridad y plazo, y la agenda con calendario y cuenta atrás.

En **economía**: el presupuesto con sus partidas, importes estimado, real y pagado, gráficas de reparto, y los proveedores con documentos adjuntos.

Y en **seguimiento**: el panel con el estado global y los avisos de tareas y pagos vencidos o próximos.

Y en el bloque destacado, las **tres asistencias con inteligencia artificial**, con ejemplos concretos: genera el listado de tareas a partir de la fecha y el número de invitados; propone la distribución de las mesas respetando los grupos y la capacidad; y reparte el presupuesto por categorías sugiriendo además proveedores. Las tres, siempre, como propuesta editable.»

Transición: «Termino con las conclusiones.»

19 — Conclusiones · 0:45

«Como conclusión, se ha construido una aplicación funcional y autocontenida que cubre el ciclo completo de organización de una boda, sin la capa comercial de las plataformas existentes.

Su aportación diferencial no es usar inteligencia artificial —eso hoy lo hace todo el mundo— sino **cómo** se usa: un patrón de propuesta verificada y bajo control del usuario, aplicado a las decisiones organizativas.

Los ocho objetivos planteados se han alcanzado, respaldados por 18 pruebas automatizadas e integración continua.

Y quiero ser honesta con la limitación: **el cliente todavía no cuenta con pruebas automatizadas propias**; su validación se apoyó en la verificación de tipos al compilar y en pruebas funcionales manuales.»

Transición: «Esa limitación enlaza directamente con la primera línea de trabajo futuro.»

20 — Líneas futuras · 0:35

(No las leas todas: agrúpalas.)

«De cara al futuro, seis líneas, por orden de prioridad.

La primera, precisamente, **cubrir el cliente con pruebas automatizadas**.

Después, en lo funcional: permitir **refinar las propuestas de la IA conversando** con el asistente en lugar de solo aceptarlas o descartarlas; una **aplicación móvil** para consultar y recibir avisos; la **exportación a PDF** de las mesas, la agenda y el presupuesto; y la **gestión compartida**, para que los dos miembros de la pareja trabajen sobre la misma boda.

Y por último, el **despliegue en producción**, que requeriría copias de seguridad y cumplimiento formal de protección de datos.»

Transición: «Y con esto termino.»

21 — Gracias · 0:10

«Muchas gracias por su atención. Quedo a su disposición para las preguntas que consideren.»

(Y te callas. No sigas hablando por nervios.)

4. Fundamentos técnicos

La parte que de verdad tienes que entender. Escrita para que puedas explicarlo con tus palabras, y en el mismo registro llano que usarás con el tribunal.

4.1 Las tres capas

- **Presentación = cliente.** Lo que corre en el navegador. React.
- **Aplicación = servidor.** Donde está la lógica y las decisiones. Node.js + Express.
- **Persistencia = base de datos.** Donde se guarda todo. PostgreSQL.

¿Por qué separarlas? Porque cada una cambia por motivos distintos: si rediseño la interfaz no toco las reglas del negocio, y si cambio la base de datos no toco la interfaz.

¿Por qué el cliente no puede guardar secretos? Porque es público: cualquiera puede abrir las herramientas del navegador y leer su código. Por eso las claves de la IA y del correo viven **solo** en el servidor.

4.2 API REST y códigos de respuesta

Una **API REST** es la lista de «puertas» del servidor. Cada recurso tiene su dirección y se opera con los verbos estándar: GET para leer, POST para crear, PUT/PATCH para modificar y DELETE para borrar.

Código	Significa	Dónde aparece en tu proyecto
200	Todo bien	Respuesta normal
400	Datos mal formados	Falla la validación de Zod
401	No autenticado	Falta la credencial o caducó
404	No encontrado	Intentar acceder a una boda ajena
409	Conflicto	Registrarse con un correo ya existente
500	Error interno	Fallo inesperado, sin filtrar detalles
502	Falla un servicio externo	La IA no responde o devuelve mal el formato

Di «no encontrado», no «404», salvo que te pregunten el código concreto.

4.3 Las dos credenciales (doble token)

	Token de acceso	Token de refresco
Dura	Minutos	Días
Para qué	Va en cada petición	Solo para pedir un acceso nuevo
Dónde vive	En el navegador	En el navegador y en la base de datos
Se puede anular	No hace falta: caduca solo	Sí — eso es cerrar sesión de verdad

El razonamiento en una frase: con un solo token tendrías que elegir entre seguridad (corto, molesto) o comodidad (largo, peligroso). Con dos tienes las dos cosas.

Renovación transparente: cuando el de acceso caduca, el cliente pide uno nuevo y reintenta la petición **una sola vez**, sin que el usuario note nada.

4.4 Aislamiento: por qué «no encontrado» y no «prohibido»

Autenticación = *quién eres*. Autorización = *qué puedes tocar*.

El riesgo es que alguien cambie el identificador en la dirección para ver datos de otro. Se ataja con **doble comprobación**: un filtro que valida la propiedad antes de nada, y otro en la propia consulta a la base de datos.

La clave que debes saber explicar: si respondiera «prohibido», estaría confirmando que ese recurso **existe**. Diciendo «no encontrado», el atacante no puede distinguir entre una boda que no existe y una que existe pero no es suya.

4.5 PostgreSQL y Prisma: quién hace qué

- **PostgreSQL** es la base de datos: guarda la información y garantiza que sea coherente.
- **Prisma** es el intermediario: traduce las órdenes de mi código al lenguaje de la base de datos y comprueba que los campos existen.

Prisma trabaja así: el modelo se declara **una vez** en un fichero; de ahí genera el código de acceso (con comprobación de tipos: si me equivoco en un campo, falla al compilar) y las **migraciones**, que son el registro versionado de cada cambio de la estructura.

Decisión clave: las garantías —un asiento no se repite, al borrar una boda no quedan restos— las impone **PostgreSQL**, no el código. Una restricción de base de datos no se puede eludir.

4.6 Zod: validar y tipar con una sola definición

Zod define qué forma deben tener los datos que entran. Su ventaja: la misma definición **valida en ejecución y genera el tipo**, así que no pueden desincronizarse. Si falla, se responde 400 con un mensaje legible.

4.7 Transacción y unicidad

- **Transacción:** operaciones que se aplican **todas o ninguna**. Se usa al dar de alta un invitado con sus acompañantes, y al aplicar la distribución de mesas de la IA.
- **Restricción de unicidad (mesa, asiento):** la impone la base de datos, así que da igual por dónde llegue la petición: es **imposible** sentar a dos personas en el mismo sitio.

4.8 La IA: el contrato

- Un modelo devuelve **texto**, y el texto no es fiable como dato.
- El **contrato** es un acuerdo cerrado sobre la forma exacta de la respuesta. Se impone al llamar (`strict`) y se **verifica** al recibir.
- Se verifica **aunque el proveedor lo garantice**: no delegas tus datos en un servicio externo.
- Modelo `gpt-4o-mini`, temperatura 0,4 (baja = consistente), límite de tokens, tiempo de espera y reintentos acotados.
- **Coste:** pago por consumo. ~0,15 \$/millón de tokens de entrada y ~0,60 \$/millón de salida. Todo el desarrollo costó unos **5 €**.

4.9 Pruebas e integración continua

- **Unitaria:** una pieza aislada, sin base de datos.
- **Integración:** el sistema de extremo a extremo, con peticiones reales.
- **Integración continua:** todo se ejecuta solo con cada cambio. Red de seguridad frente a **regresiones** (romper sin querer algo que funcionaba).

4.10 El stack y su porqué, en una tabla

Tecnología	Qué es	Por qué esa
TypeScript	JavaScript con tipos	Avisa de errores al escribir, no en producción
React	Interfaz por piezas reutilizables	Modelo simple, ecosistema amplio
Vite	Compila y sirve en desarrollo	Arranque y recarga casi instantáneos
Tailwind	Estilos por clases predefinidas	Coherencia visual sin CSS a medida
shadcn/ui	Componentes accesibles copiados al proyecto	Adaptables, sin dependencia pesada
Node.js	JavaScript en el servidor	Mismo lenguaje en ambos lados
Express	Estructura mínima para la API	Maduro y simple; NestJS sería excesivo
Zod	Define la forma de los datos de entrada	Valida y tipa con una sola definición
Prisma	Intermediario con la base de datos	Una sola fuente de verdad; migraciones
PostgreSQL	La base de datos	Integridad referencial y tipos avanzados

5. Preguntas

Banco completo con **80 preguntas**: [preguntas-defensa.pdf](#) . Aquí, las que debes dominar sí o sí.

1. **Resumen del trabajo** en un minuto.
2. **Qué aporta** frente a Bodas.net / The Knot / Zola.
3. **Las tres capas** y por qué están separadas.
4. **Cómo garantizas el aislamiento** (doble comprobación, «no encontrado» en vez de «prohibido»).
5. **El flujo de la IA**: contexto → forma exigida → validación → propuesta → confirmación.
6. **Por qué la IA no escribe** directamente en la base de datos.
7. **Cómo evitas** que dos invitados ocupen el mismo asiento (lo impide la base de datos).
8. **Por qué dos tokens** y no uno.
9. **Qué datos se envían a OpenAI** y qué implicación de privacidad tiene.
10. **Por qué no hay pruebas del cliente**, reconociéndolo como limitación.

Si no sabes algo: dilo. *«No lo había considerado desde ese ángulo; lo que sí puedo decirle es que...»*. Si te repiten una crítica, **dales la razón**. Nunca te inventes un dato.

6. Checklist del día

Antes de entrar

- [] Presentación en **PDF** en el portátil **y** en un USB.
- [] Si hay vídeo: **incrustado o en el mismo USB**, y probado en ese portátil.
- [] Probar proyector y resolución.
- [] Portátil cargado + cargador. Agua.

Lenguaje no verbal *(de la guía de tu tutor)*

- **Empieza pidiendo permiso**. Y pídelo otra vez antes del vídeo, si lo pones.
- **Ojos**: al tribunal, alternando. Si te bloquea, al entrecejo. **Nunca a la pantalla**.
- **Manos**: un bolígrafo o el pasador, para no gesticular de más.
- **Pies**: sin moverte en exceso, pero cambia de posición. **Nunca des la espalda**.
- **Puntero**: evita el láser. **Señala con la mano**.
- **Truco Jedi**: donde mires tú, mirará el tribunal.
- **No leas la pantalla y no corras**.

Durante las preguntas

- Escucha la pregunta entera. Si no la entiendes, pide que te la repitan.
- Respuestas cortas y concretas. Si quieren más, preguntarán.

- Si te critican algo, agradécelo y dales la razón.

7. Plan de estudio

Cuándo	Qué hacer
Días 1-2	Parte 4 (Fundamentos) entera, dos veces. Entender, no memorizar.
Día 3	Explicar el proyecto en voz alta a alguien no técnico, en 3 minutos.
Día 4	<code>preguntas-defensa.pdf</code> . Responder en voz alta con tus palabras.
Día 5	Ensayo cronometrado. Objetivo: 20:00 . Apunta dónde te atascas.
Día 6	Segundo ensayo, centrado en las diapositivas 6-13 (las de diseño).
Víspera	Repaso de la parte 1 y el checklist. Dormir bien.

Prueba de fuego: si puedes explicar sin mirar nada *por qué hay dos tokens, por qué la IA no escribe en la base de datos, por qué responde «no encontrado» y qué diferencia hay entre Prisma y PostgreSQL*, estás lista.

Mucha suerte. Eres quien mejor conoce este trabajo en esa sala.